

Низькорівневе програмування та програмування мікроконтролерів

З використанням мови програмування **Assembler**
Цикли та циклічні операції

класрум **im3ozqq4**

Питання

Ми звикли, що в усіх мовах програмування присутні конструкції

```
for (int i = 0; i < 10; i++) {  
    ...  
}
```

```
while (...) {  
    ...  
}
```

Що робити з асемблером

Технічно, маємо блок,
Який потрібно повторювати
Або декілька разів,
Або на кожному кроці перевіряти якусь умову

Враховуючи не дуже велику кількість типів даних,
ми можемо перевіряти наступні речі:

```
x == 0
x != 0
x > 0
x < 0
x % 2 == 0
```


Або в загальному випадку

$x == y$

$x != y$

$x > y$

$x >= y$

$x < y$

$x <= y$

Для цього ми використовуємо математичні операції, такі як **add**, **sub**, ... логічні **and**, **or**, **xor**, **not**, ...

А також спеціальні операції порівняння **test**, **cmp**.

І далі будемо використовувати команди умовного та безумовного переходу для композиції циклів

```
mov bl, 251  
mov cl, 3  
add bl, cl
```

...	CF = 0
...	
...	
...	

...	CF = 1
...	
...	
...	

Команди переходу

Mnemonic	Jumps if...	Flag test
J0	overflow	OF = 1
JNO	no overflow	OF = 0
JS	sign (negative)	SF = 1
JNS	not sign (non-negative)	SF = 0
JZ	zero / equal	ZF = 1
JNZ	not zero / not equal	ZF = 0
JC	carry / below (unsigned)	CF = 1
JNC	not carry / not below (unsigned)	CF = 0
JP	parity even	PF = 1
JNP	parity odd	PF = 0

Команди переходу

Операція	Unsigned		Signed	
==	JE / JZ	JNE / JNZ	JE / JZ	JNE / JNZ
!=	JNE / JNZ	JE / JZ	JNE / JNZ	JE / JZ
>	JA (Above)	JBE (Below or Equal)	JG (Greater)	JLE (Less or Equal)
>=	JAE / JNB (Above or Equal)	JB (Below)	JGE (Greater or Equal)	JL (Less)
<	JB (Below)	JAE / JNB (Above or Equal)	JL (Less)	JGE (Greater or Equal)
<=	JBE (Below or Equal)	JA (Above)	JLE (Less or Equal)	JG (Greater)

Працюють вони звісно
однаковим чином, але ми
розберемо основні патерни
використання

повтор певну кількість разів

```
mov    ecx, 10  
start:  
    ; ... обробка ...  
    dec    ecx  
    jnz    start
```

Зазвичай в асемблері ми рахуємо від **N** до **0**

повтор певну кількість разів

```
    mov     ecx, ...  
    jecxz   exit  
start:  
    ; ...  обробка ...  
    dec     ecx  
    jnz     start  
exit:
```

Додаткова перевірка, якщо CX = 0

Фактично, на кожному кроці
ми робимо **dec cx**
і потім робим **jmp**

Команда `loop` (використання регістру `CX/ECX`)

```
go:
; .....
; ... тіло ...
; .....
dec     ecx
jnz     go
```

```
go:
; .....
; ... тіло ...
; .....
loopnz  go
```

Сума елементів масиву

```
mov     esi, aa
mov     ecx, 4

test    ecx, ecx
jz      done

xor     eax, eax
sum:
add     eax, [esi]
add     esi, 4
loop    sum
done:

aa      dd 1,2,3,4
```


LOOP. Можливі варіації синтаксису

Мнемоніка	Умова переходу (після декременту)	Читає прапори	Модифікує прапори
LOOP	--ECX != 0	—	не змінює
LOOPZ	--ECX != 0 і ZF == 1	ZF	не змінює
LOOPNZ	--ECX != 0 і ZF == 0	ZF	не змінює

кількість встановлених бітів EAX

```
mov    eax, 0001100111001111b
xor     ecx, ecx
go:
test    eax, eax
jz      done
shr     eax, 1
adc     ecx, 0
jmp     go
done:
```


do ... while (min 1 iteration)

```
mov     ecx, 5  
go:  
    ; ... тіло ...  
    dec     ecx  
    jnz     go
```

Команда **rep***
повтор операції
відповідно значенню в реєстрі **CX***

заповнення буфера значенням з регістру AL

```
mov edi, buf
xor al, 0x13
cld
rep stosb

buf db 'hello'
len equ $-buf
```

заповнення буфера значенням з регістру AL

```
mov    edi, buf
xor     al, 0x13
cld
rep     stosb

buf    db    'hello'
len    equ   $-buf
```

```
stosb
stosw
stosd
stosq
```


копіювання пам'яті

```
mov    ecx, len
mov    esi, a1
mov    edi, a2
cld
rep    movsb

a1      db    'hello'
len      =    $-a1
a2      db    len dup (?)
```

копіювання пам'яті

```
mov    ecx, len
mov    esi, a1
mov    edi, a2
cld
rep    movsb

a1      db    'hello'
len     =     $-a1
a2      db    len dup (?)
```

```
movsb
movsw
movsd
movsq
```


cld / std - напрямок

ПОШУК байта

шукаємо 'l' в [EDI .. EDI+ECX),

ZF=1 - found

ZF=0 - not found

```
mov    al, 'l'
mov    edi, a1
mov    ecx, len
repne scasb
```

```
a1      db    'hello'
len      =    $-a1
```


пошук байта

шукаємо 'l' в [EDI .. EDI+ECX)

```
mov    al, 'l'  
mov    edi, a1  
mov    ecx, len  
repne scasb
```

```
a1      db  'hello'  
len     =  $-a1
```

```
scasb  
scasw  
scasd  
scasq
```

Порівняння рядків

```
mov     esi, a1
mov     edi, a2
mov     ecx, len
cld
repe    cmpsb
jne     diff

same:
...
jmp     exit

diff:
...

exit:

a1      db    'hello'
len     =     $ - a1
a2      db    'hellp'
```


Порівняння рядків

```
mov     esi, a1
mov     edi, a2
mov     ecx, len
cld
repe    cmpsb
jne     diff

same:
...
jmp     exit

diff:
...

exit:

a1      db    'hello'
len     =    $ - a1
a2      db    'hellp'
```

```
cmpsb
cmpsw
cmpsd
cmpsq
```

REP* – повтор операції

Префікс	Умова повторення (після кожної ітерації)	Коментар
REP	--ECX != 0	Класичне повторення ECX разів (копіювання, запис, введення/виведення).
REPE	--ECX != 0 і ZF == 1	Повторює поки «співпадає» (ZF=1), використовується для CMPS / SCAS.
REPNE	--ECX != 0 і ZF == 0	Повторює поки «не співпадає» (ZF=0), використовується для CMPS / SCAS.

Можливі операції (аргумент REP)

Мнемоніка	Що відбувається за 1 ітерацію	Прапори
MOVSw	Копіює [DS:SI] → [ES:DI], зсуває SI, DI	не чіпає
STOSw	Записує AX → [ES:DI], зсуває DI	не чіпає
LODSw	Читає [DS:SI] → AX, зсуває SI	не чіпає
CMPSw	Порівнює [DS:SI] з [ES:DI], зсуває SI, DI	встановлює ZF, SF, OF, CF, AF, PF
SCASw	Порівнює AX з [ES:DI], зсуває DI	встановлює як для SUB
INSw	IN з порту DX → [ES:DI], зсуває DI	не чіпає
OUTSw	Читає [DS:SI] → OUT у порт DX, зсуває SI	не чіпає
NOP	спецвипадок (REP NOP = PAUSE)	не чіпає

вкладені цикли (сума матриці)

```
table db 1, 2, 3, 4  
      db 5, 6, 7, 8  
      db 9, 10, 11, 12
```


Використаємо можливості компілятора

```
table    db 1, 2, 3, 4
n_cols   = $ - table
|         |         |
|         |         | db 5, 6, 7, 8
|         |         | db 9,10,11,12
n_rows   = ($ - table) / n_cols
```

Підготовчі операції

```
xor    eax, eax  
mov    esi, table
```


Цикл по першому рядку

```
mov  cx, n_cols  
col1:  
    add  al, [esi]  
    inc  esi  
    loop col1
```

Цикл по колонкам

```
mov cx, n_rows  
row:  
    ...  
loop row
```


Комбінуємо цикли

```
    mov    cx, n_rows
row:
    push   cx
    mov    cx, n_cols
col:
    add    al, [esi]
    inc    esi
    loop   col

    pop    cx
    loop   row
```

Збираємо
все
разом

```
xor    eax, eax
mov    esi, table

mov    cx, n_rows

row:
  push  cx
  mov   cx, n_cols

  col:
    add  al, [esi]
    inc  esi
    loop col

  pop   cx
  loop  row
```


Notes

Notes

Notes

Notes